6.2 Neural Network with Julia

As was seen in the previous notebook, a Neural Network can get very big and very complex to define with lots of parameters.

In Julia, there are libraries that can take care of all the Neural Network definitions and setup. Here we will use the `Lux.jl` library in Julia to define, setup and tune the Neural Network

In [2]:
```
using Zygote
using Lux
using Random
using Distributions
using CairoMakie
using Optimisers
using Optimization
using OptimizationOptimisers
using ComponentArrays
```

In this notebook, we will see how we can use the `Lux()` package in Julia to set up and use Neural Networks. Let's initially define a simple Neural Network in Lux

$$NN(x) = Wx + b$$

with one input,x and one output, $NN(x)$ with weight and bias W and b

In [3]:
```
model01 = Chain(Dense(1 => 1))
```

```
Chain(
    layer_1 = Dense(1 => 1),                    # 2 parameters
)         # Total: 2 parameters,
          #        plus 0 states.
```

In general, you are usually encouraged to use random number generator to initialize the values of the weight and bias.

In [4]:
```
# Initialize the model weights and state
rng = Random.default_rng()
parameters, layer_states = Lux.setup(rng, model01)
```

```
((layer_1 = (weight = Float32[0.6798217;;], bias = Float32[0.9002472]),),
(layer_1 = NamedTuple(),))
```

If you want to, you can explicitly change the value of W to 2.0 and b to 1.0 using the code below

In [5]:
```
parameters.layer_1.weight.=Float32(2.0)
parameters.layer_1.bias.=Float32(1.0)
```

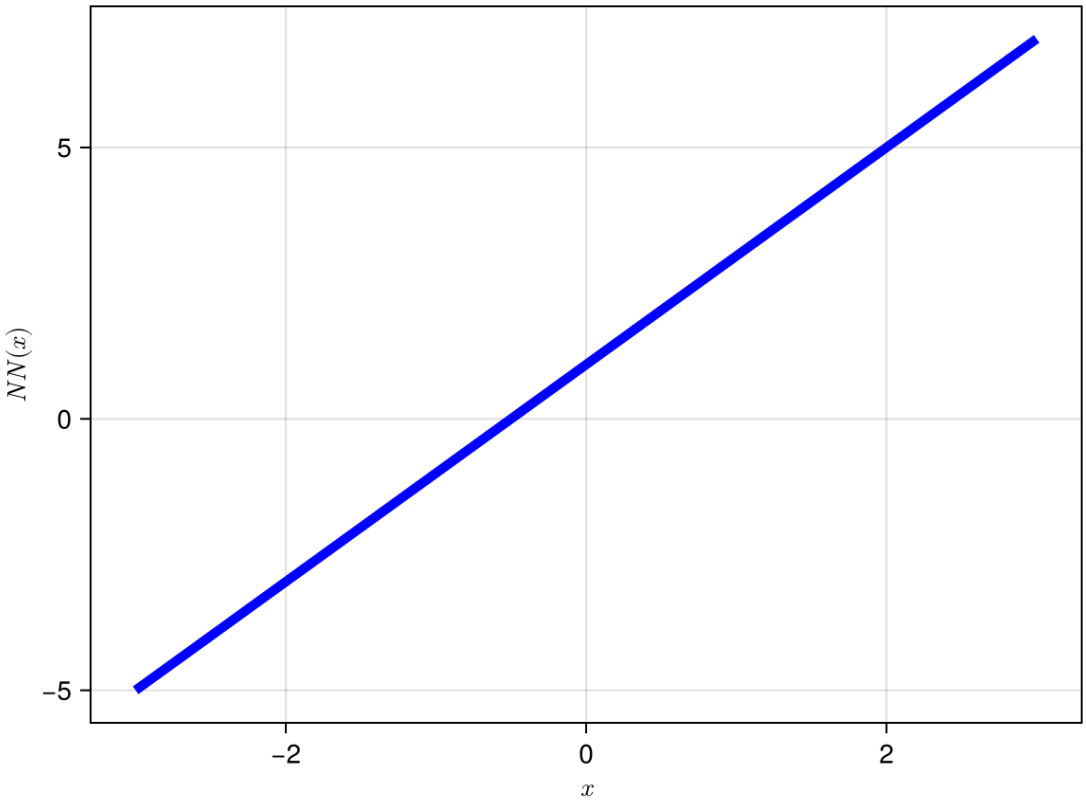
```
1-element Vector{Float32}:
 1.0
```

Note that by default, the Neural Network is defined as single precision Float32 type. This is because single precision is accepted to be the optimal balance between precision for accurate training and computational speed/memory efficiency on GPUs.

Let's plot a graph of the Neural Network to see how it looks like

In [6]:
```
xgrid=-3.0:0.01:3.0
y_prediction,_=model01(xgrid',parameters, layer_states)
fig = Figure()
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"x", ylabel = L"NN(x)")
lines!(ax,xgrid,y_prediction[:];color=:blue,label="prediction",linewidth=
save("../figures/06p02NeuraNetworkWithJulia01.svg",fig)
fig
```

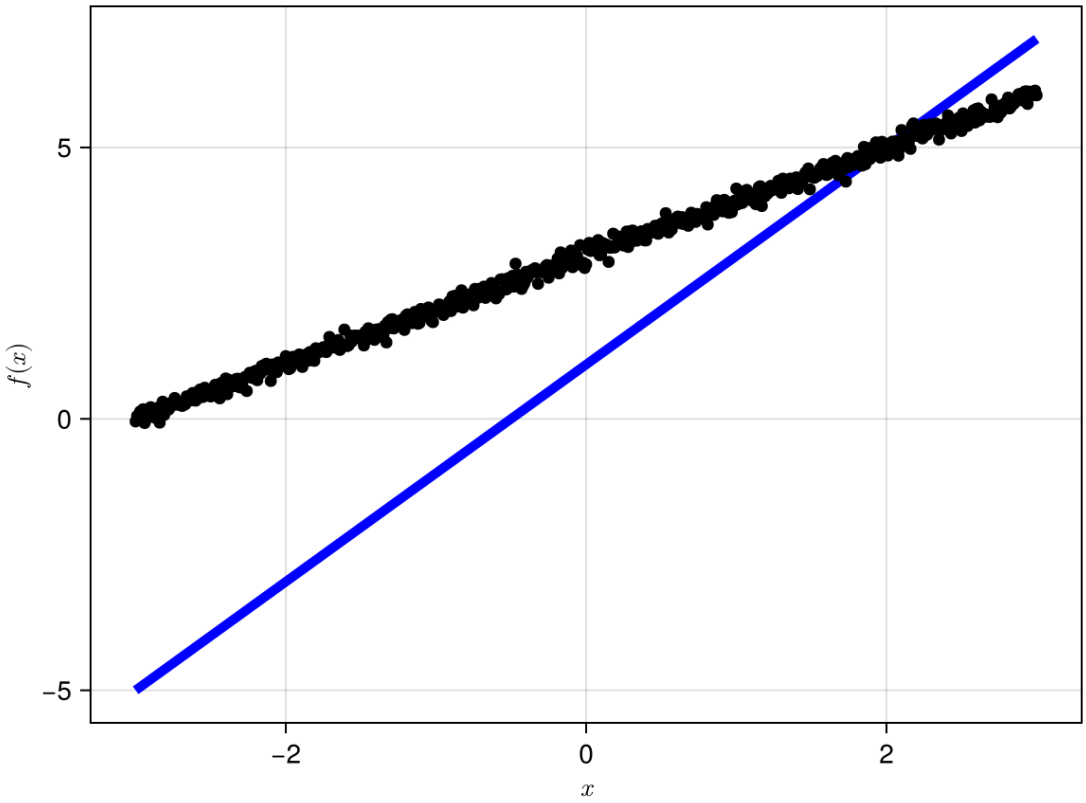
```
┌ Warning: Mixed-Precision `matmul_cpu_fallback!` detected and Octavian.jl
cannot be used for this set of inputs (C [Matrix{Float64}]: A [Matrix{Floa
t32}] x B [LinearAlgebra.Adjoint{Float64, StepRangeLen{Float64, Base.Twice
Precision{Float64}, Base.TwicePrecision{Float64}, Int64}}]). Falling back
to generic implementation. This may be slow.
└ @ LuxLib.Impl /Users/asho/.julia/packages/LuxLib/zPBrt/src/impl/matmul.j
l:194
```



Class Demo 01: Define and plot the Neural Network $NN(x) = \sin(3x + 4)$ for $x \in [-2, 3]$

Let's now generate some data and try to train this Neural Network so that it fits the data. We will artificially generate some data with the function $y(x) = x + 3$. Let's see our current Neural Network prediction with this data

In [7]:
```
xgrid=collect(-3.0:0.01:3.0)
N_SAMPLES=length(xgrid)

#y_prediction,_=model01(reshape(xgrid,(1,N_SAMPLES)),parameters, layer_st
y_prediction,_=model01(xgrid',parameters, layer_states)
y_data=xgrid.+3+0.1*randn(N_SAMPLES)

fig = Figure()
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"x", ylabel = L"f(x)")
lines!(ax,xgrid,y_prediction[:];color=:blue,label="prediction",linewidth=
scatter!(ax,xgrid,y_data[:];color=:black,label="data")
fig
```



Now we try to train this Neural Network. As usual, first define the loss function

In [8]:
```
# loss function
function loss_fn(p, ls)
    y_prediction, _= model01(xgrid', p, ls)
    loss = 0.5 * mean((y_prediction[:] .- y_data).^2)
    return loss
end
```

```
loss_fn (generic function with 1 method)
```

In [9]:
```
f_loss=(ps) -> loss_fn(ps,layer_states)
```

```
#15 (generic function with 1 method)
```

We can use `Zygote()` to find the gradient of the loss function

In [10]:
```
Zygote.gradient(f_loss,parameters)[1]
```

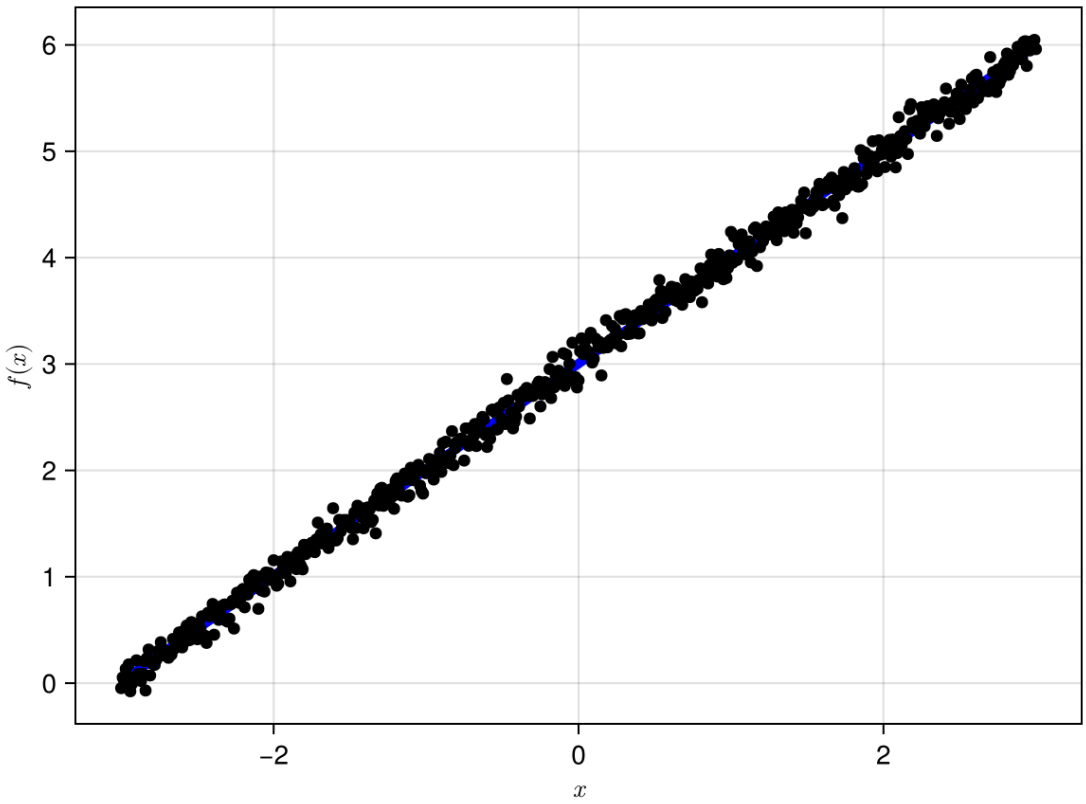
```
(layer_1 = (weight = Float32[3.0189338;;], bias = Float32[-2.0037873]),)
```

Now we have the gradient, we can use standard gradient descent to train the neural network so that it best fits the data

In [11]:
```
α=0.01

new_weight=parameters.layer_1.weight[1]
new_bias=parameters.layer_1.bias[1]

for i=1:5000
    grads=Zygote.gradient(f_loss,parameters)[1]
    new_weight-=α*grads.layer_1.weight[1]
    new_bias-=α*grads.layer_1.bias[1]

    parameters.layer_1.weight.=Float32(new_weight)
    parameters.layer_1.bias.=Float32(new_bias)
end
```

Plot and compare

In [12]:
```
y_prediction,_=model01(xgrid',parameters, layer_states)


fig = Figure()
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"x", ylabel = L"f(x)")
lines!(ax,xgrid[:],y_prediction[:];color=:blue,label="prediction",linewid
scatter!(ax,xgrid[:],y_data[:];color=:black,label="data")
fig
```



In [13]:
```
parameters.layer_1.bias[1]
```

```
3.0037873f0
```

In [14]:
```
parameters.layer_1.weight[1]
```

```
0.997032f0
```

As you can see, the values of the weights and biases of the parameters are as expected

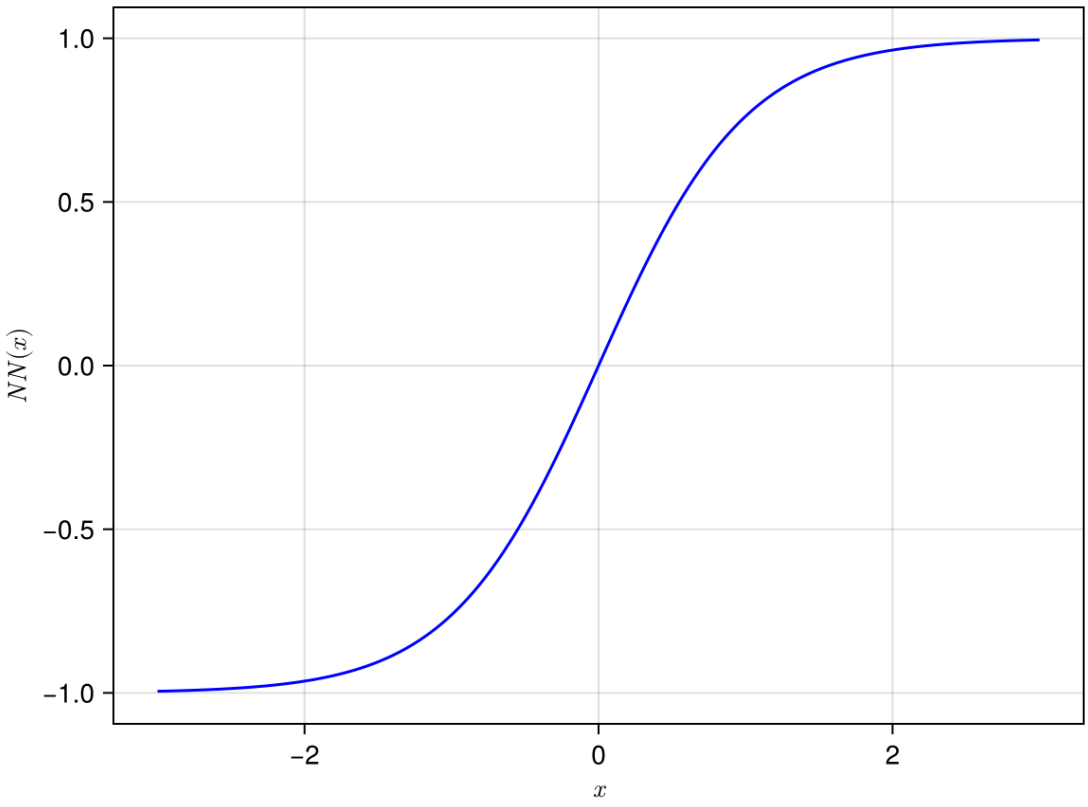
---

We can also introduce activation layers to make the Neural Network more complex and can better approximate nonlinear functions. The code below defines a neural network with a $\tanh()$ activation layer. The Neural Network defined below looks like

$$NN(x) = \tanh(Wx + b)$$

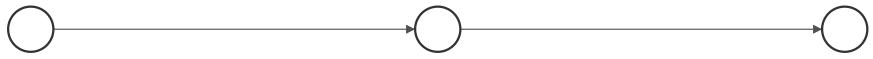
It has one input and one output.

In [15]:
```
model02 = Chain(Dense(1 => 1,tanh))
parameters.layer_1.weight.=Float32(1.0) # Change the weight to 1.0
parameters.layer_1.bias.=Float32(0.0) # Change the bias to 0.0

y_prediction,_=model02(xgrid',parameters, layer_states) # predict the out

fig = Figure()
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"x", ylabel = L"NN(x)")
lines!(ax,xgrid[:],y_prediction[:];color=:blue,label="prediction")
fig
```



We will now define a Neural Network with one input layer, one output and one hidden layer. There is we will use a $\tanh()$ activation function between the input and hidden

layer. Visually, the Neural Network looks like



Input Layer $\in \mathbb{R}^1$ Hidden Layer $\in \mathbb{R}^1$ Output Layer $\in \mathbb{R}^1$

From the previous notebook, we have established that this Neural Network can be mathematically expressed as

$$NN(x) = W_2 \tanh(W_1 x + b_1) + b_2$$

In [16]:
```
model03 = Chain(Dense(1 => 1,tanh),
Dense(1=>1))
```

```
Chain(
    layer_1 = Dense(1 => 1, tanh),              # 2 parameters
    layer_2 = Dense(1 => 1),                    # 2 parameters
)         # Total: 4 parameters,
          #        plus 0 states.
```

Randomly initialise the weights and biases of the Neural network

In [17]:
```
# Initialize the model weights and state
parameters03, layer_states03 = Lux.setup(rng, model03)
```

```
((layer_1 = (weight = Float32[1.4448158;;], bias = Float32[-0.10772669]),
layer_2 = (weight = Float32[-0.3272207;;], bias = Float32[-0.70124984])),
(layer_1 = NamedTuple(), layer_2 = NamedTuple()))
```

You do not need to do this but you can set the weights and biases to be $W_1 = 1$, $W_2 = 1$,$b_1 = 0$ and $b_2 = 0$.

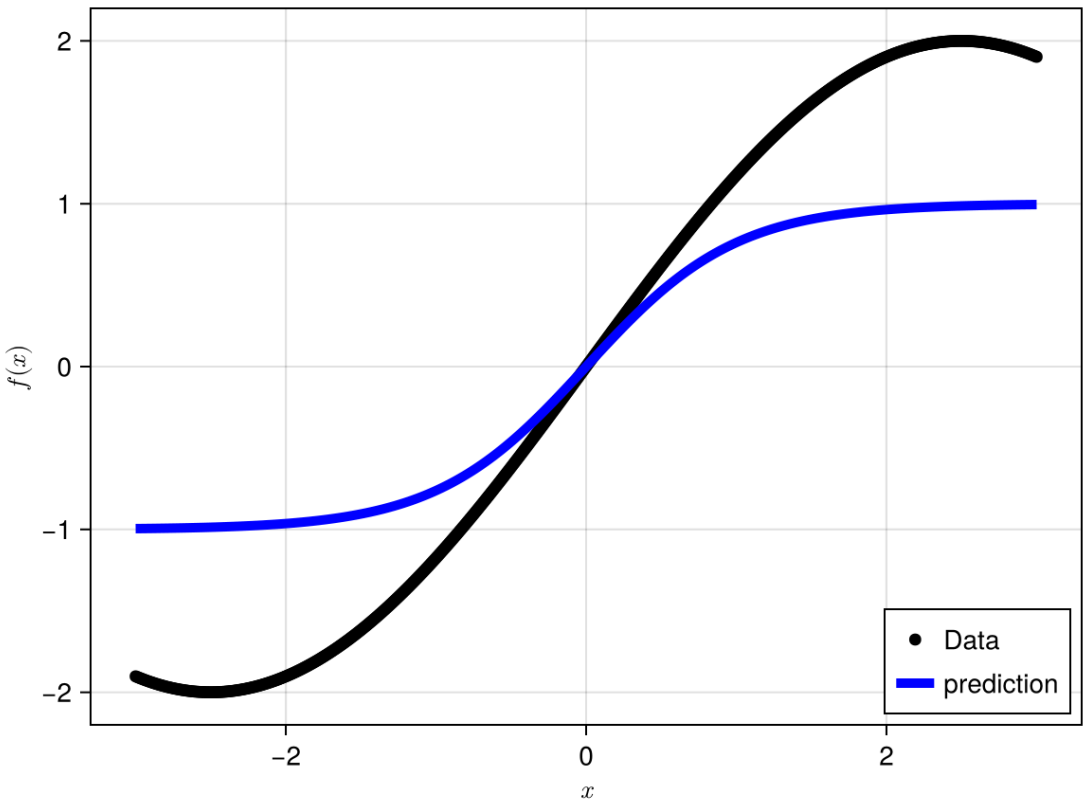
In [18]:
```
parameters03.layer_1.weight.=Float32(1.0)
parameters03.layer_2.weight.=Float32(1.0)
parameters03.layer_1.bias.=Float32(0.0)
parameters03.layer_2.bias.=Float32(0.0)
```

```
1-element Vector{Float32}:
 0.0
```

Let's generate some data and try to fit this Neural Network to the data

In [19]:
```
g(x)=2.0sin(pi *x/5 )
data03=g.(xgrid)


y_prediction,_=model03(xgrid',parameters03, layer_states03) # predict the


fig = Figure()
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"x", ylabel = L"f(x)")
scatter!(ax,xgrid,data03;color=:black,label="Data")
lines!(ax,xgrid,y_prediction[:];linewidth=5,color=:blue,label="prediction

axislegend(ax,  position = :rb,orientation = :vertical)
```

```
fig
```



We will now define the loss function

In [ ]:
```
# The loss function
function loss_fn03(p, ls)
    y_prediction, _= model03(xgrid', p, ls)
    loss = 0.5 * mean((y_prediction[:] .- data03).^2)
    return loss
end
```

```
loss_fn03 (generic function with 1 method)
```

In [21]:
```
f03_loss=(ps) -> loss_fn03(ps,layer_states03)[1]
```

```
#17 (generic function with 1 method)
```

In [22]:
```
f03_loss(parameters03)
```

```
0.25464908237964395
```

We can use the `Zygote()` package to find calculate the gradient of the loss function

In [23]:
```
grads=Zygote.gradient(f03_loss,parameters03)[1]
```

```
(layer_1 = (weight = Float32[-0.10721234;;], bias = Float32[-2.3264607f-1
7]), layer_2 = (weight = Float32[-0.56792516;;], bias = Float32[-3.1658703
f-17]))
```

Since we know how to calculate the gradient, we can use standard gradient descent to find the line of best fit

In [24]:
```
new_weight_l1=parameters03.layer_1.weight[1]
new_bias_l1=parameters03.layer_1.bias[1]
new_weight_l2=parameters03.layer_2.weight[1]
new_bias_l2=parameters03.layer_2.bias[1]

α=0.2

for i=1:500
    grads=Zygote.gradient(f03_loss,parameters03)[1]
    new_weight_l1-=α*grads.layer_1.weight[1]
    new_bias_l1-=α*grads.layer_1.bias[1]
    new_weight_l2-=α*grads.layer_2.weight[1]
    new_bias_l2-=α*grads.layer_2.bias[1]

    parameters03.layer_1.weight.=Float32(new_weight_l1)
    parameters03.layer_1.bias.=Float32(new_bias_l1)
    parameters03.layer_2.weight.=Float32(new_weight_l2)
    parameters03.layer_2.bias.=Float32(new_bias_l2)

    println(f03_loss(parameters03))
end
```

```
0.19241417605960304
0.14620459136559114
0.11214481123268719
0.08717325297509951
0.06892620422522926
0.0556112822929430 6
0.045888798363561835
0.03876824345121391
0.03352384122314505
0.029627426975085847
0.026696825395886293
0.024456793898072586
0.02270993856445393
0.02131513806748626
0.02017185505097349
0.019208685155886623
0.018375052616525007
0.017635249060930593
0.016964175803340593
0.01634423196189717
0.015763141573507854
0.015212401044487305
0.014686136263476552
0.014180344449122403
0.013692309791279651
0.013222020953445252 4
0.012762848993512741
0.012319433803634136
0.011889460805531276
0.011472589767450308
0.011068610204517533
0.010677365178150518
0.010298725577200567
0.009932581413938865
0.009578810193236318
0.00923728069546794
0.008907837196029217
0.008590310813068934
0.008284507655203088
0.007990208561996646
0.007707179318916679
0.00743516316762422
0.007173892949543088 5
0.006923084965599306
0.006682445817382202
0.006451670788789293
0.006230450111664911
0.006018472488571302
0.005815422756799123
0.005620984988923092
0.005434845699192337
0.005256693367356539
0.005086220447154446
0.004923127406003933
0.004767114196436387
0.004617892807662616
0.004475183500992832
0.004338709847425563
0.004208210502266202
0.004083424388752215
```

```
0.003964103376555651
0.0038500108835465173
0.0037409135387213655
0.003636588275081237
0.003536825589142099
0.0034414188615958245
0.003350171598590526
0.0032628970894021587
0.0031794129061319875
0.0030995481903074965
0.0030231392467235507
0.002950028153950165
0.0028800645230115884
0.002813104674572453
0.0027490118279253715
0.00268765855012615
0.002628915040141597
0.00257263751595584
0.0025187939355796156
0.0024671994115379593
0.00241777107863514
0.002370413518739105
0.0023250368483785783
0.002281547956771778
0.00223986400475167 5
0.0021999040056328 69
0.002161590339472623
0.00212484961892338 1
0.00208961441775790 3
0.0020558151969732436
0.002023388647182906
0.00199227822847854 5
0.0019624218062797745
0.0019337679832926351
0.0019062602458531119
0.0018798540227108796
0.0018544974081147435
0.0018301478378807605
0.0018067621999876132
0.00178429596577655 8
0.0017627147035902259
0.0017419766150583243
0.00172204820604207 4
0.0017028945328000497
0.00168448353186466 6
0.00166678290691440 4
0.00164976580492219 6
0.00163339988963254 7
0.0016176611187071724
0.0016025231138858424
0.00158796034582474 1
0.0015739509202286968
0.0015604687071045795
0.0015474957000705877
0.0015350114070963419
0.0015229923251182413
0.0015114245774744957
0.00150028548298287 3
0.0014895613621450307
0.0014792345640720083
```

```
0.0014692885449261008
0.00145970808710713
0.0014504807174926515
0.0014415911292989497
0.0014330252385256518
0.0014247726381052478
0.00141416819116520908
0.0014091548658341066
0.0014017670097309625
0.0013946455161714195
0.0013877793503327303
0.0013811612702667344
0.0013747788612221586
0.0013686249500288368
0.00136262689706721063
0.0013569654305081076
0.0013514451858659244
0.0013461197062905612
0.0013409817608398958
0.0013360253443600904
0.0013312431736521916
0.0013266287104601816
0.0013221748650121268
0.0013178777072780199
0.0013137294872702548
0.0013097266020159109
0.0013058621758157076
0.0013021309059792444
0.0012985297603014842
0.0012950520481623028
0.0012916951330009252
0.0012884528299045425
0.0012853216962369976
0.00128229850265719 8
0.0012793783419688 29
0.0012765573670693484
0.0012738335595904 99
0.0012712018054552884
0.0012686593358733 15
0.0012662034547120556
0.0012638302774766997
0.0012615375247778345
0.0012593219914956938
0.0012571815483167 39
0.0012551126313315678
0.0012531131800470954
0.0012511811842158662
0.0012493135619224289
0.0012475082674086205
0.0012457635913786 83
0.0012440767046740 18
0.0012424465100845355
0.0012408701463483 28
0.0012393461469740507
0.0012378724512398154
0.0012364476043394547
0.0012350703510878398
0.0012337381880413 81
0.0012324503197051694
0.0012312046088149 29
```

```
0.00122999983353508
0.00122834802444813
0.00122777079529301707
0.00122661770497887 12
0.00122563526911953
0.00122454397507327 39
0.00122355758850841 12
0.00122260353355780 32
0.00122168036205829 15
0.00122078736728739 07
0.00121992317859064 07
0.00121908736198939 87
0.00121827845554469 44
0.00121749606510121 06
0.0012167391308410 91
0.0012160064533956 39
0.00121529737552156 26
0.0012146113686618 04
0.00121394732081295 34
0.0012133050504299 04
0.0012126833003641 07
0.00121208135236009 51
0.0012114988260743 17
0.001210935084723 52
0.00121038966269433 33
0.00120986149948130 35
0.00120935030002218 08
0.00120885564581664 08
0.00120837655094267 18
0.00120791285202535 33
0.00120746407742442 66
0.00120702956213462 15
0.00120660900275978 96
0.0012062016372170 74
0.00120580743039516 45
0.0012054257784854 22
0.0012050562916993 64
0.00120469844439027 28
0.00120435183470791 56
0.00120401649252625 94
0.00120369158534556 65
0.00120337707861643 74
0.00120307246888500 67
0.00120277772520870 12
0.00120249212167724 52
0.00120221536951373 45
0.00120194776046241 25
0.00120168834080897 87
0.00120143707641798 75
0.0012011931938047 82 58
0.00120095833150989 46
0.00120073018516606 66
0.0012005091914179 07
0.00120029514736381 07
0.00120008791694470 84
0.00119988727355739 52
0.0011996926593950 34
0.0011995045043277 97 23
0.00119932190232371 63
0.00119914521297602 22
```

```
0.0011989741501638813
0.0011988081819113668
0.0011986476808822277
0.0011984920294676174
0.0011983414400857891
0.0011981953905165056
0.001198053971742539
0.0011979169046510867
0.0011977842022375897
0.001197655620956856
0.0011975310392878613
0.0011974103388440154
0.0011972934506273131
0.0011971802829008325
0.001197070401228278
0.0011969641973124607
0.001196861099350689
0.0011967613452896994
0.0011966645716045912
0.0011965709665346656
0.0011964801585778223
0.001196392238275503
0.001196306984049361
0.0011962244642939137
0.0011961444844454858
0.0011960668402670196
0.0011959918333153843
0.001195918944423805
0.0011958484529060907
0.001195780062535367
0.0011957138421128373
0.0011956497393224113
0.001195587573085693
0.0011955273154791036
0.0011954688893232878
0.0011954122683502813
0.0011953574101558689
0.0011953043462228582
0.0011952527393452412
0.0011952028778954077
0.001195154473832984
0.001195107622052191
0.0011950621883267578
0.0011950182067204276
0.001194975562465354
0.0011949342109598747
0.001194894135924277
0.0011948552808718582
0.0011948176437372158
0.0011947811698039313
0.0011947458444409207
0.0011947115439209183
0.0011946784107718807
0.0011946461834839923
0.0011946149797127526
0.001194584774806775
0.0011945554785478198
0.001194527070185695
0.0011944995821125525
0.0011944728657850461
```

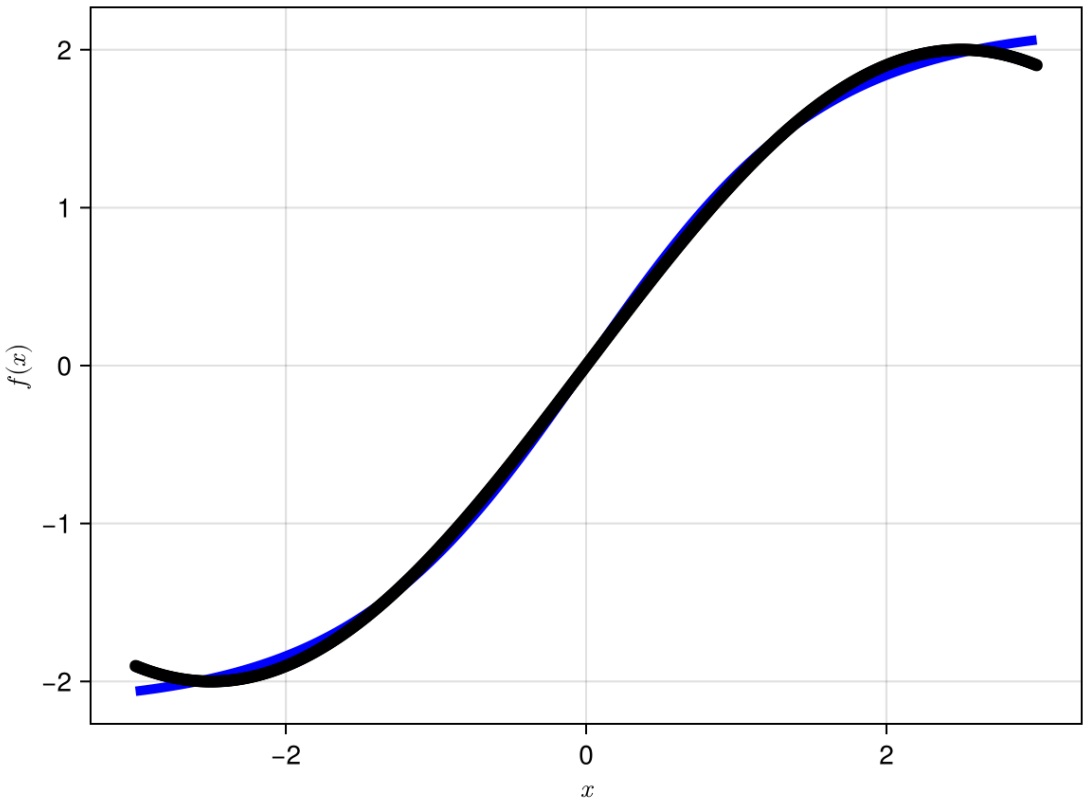
```
0.001194447028928762
0.0011944219886285503
0.0011943976556449195
0.0011943741332881867
0.001194351284673863
0.00119432921997335 1
0.0011943077863503081
0.0011942869701733509
0.0011942668127652824
0.00119424730772298
0.0011942283675047593
0.0011942100411084944
0.001194192253080527
0.0011941750002694243
0.0011941583116550508
0.001194142124202172
0.001194126426499841
0.0011941111907351433
0.0011940964861415968
0.0011940821744403623
0.0011940682932875416
0.0011940549009731084
0.0011940418735598955
0.001194029254552754
0.0011940170268242534
0.0011940051391283958
0.0011939936743751195
0.0011939825327693316
0.001193971700477527
0.0011939612174489231
0.0011939510751684365
0.0011939412652875894
0.001193931691015877
0.001193922435588516
0.0011939134846520047
0.0011939047946567733
0.00119396359846504
0.0011938882094673856
0.0011938803019428416
0.0011938726259152627
0.00119386518205093 1
0.0011938579479314967
0.0011938509738651862
0.0011938441782681343
0.0011938376149983217
0.0011938312579979726
0.0011938250614427885
0.0011938190873857244
0.00119383813300484098
0.0011938076622732988
0.0011938021935530992
0.0011937969002215674
0.0011937917732243203
0.00119378681333910 5
0.0011937820118056313
0.0011937773338002362
0.0011937728075254546
0.0011937683992170325
0.0011937641657883493
0.00119376003970314 7
```

```
0.0011937560274491114
0.0011937521507632554
0.0011937483979810736
0.0011937447508691427
0.0011937411992265423
0.001193737798225673
0.0011937344611331424
0.0011937312388675114
0.001193728132227605
0.001193725088233651
0.0011937221689922294
0.0011937193126096014
0.0011937165588509018
0.00119371388152976
0.0011937113021506713
0.0011937087725979476
0.0011937063562557988
0.001193703993289478
0.0011937016918379076
0.0011936994815686438
0.0011936973115791044
0.0011936952260707208
0.001193693219667978
0.0011936912559206666
0.0011936893482266486
0.001193687495377752
0.0011936857182001076
0.00119368397672871
0.001193682305014925
0.0011936806644436621
0.0011936790934755981
0.0011936775696850478
0.0011936760770857111
0.0011936746446701478
0.0011936732418012556
0.001193671882361321
0.001193670584190999
0.0011936693082763047
0.0011936680751421127
0.0011936668790819903
0.0011936657083667982
0.0011936645885874126
0.0011936635076796383
0.0011936624477019925
0.0011936614208305004
0.0011936604283944768
0.0011936594695309863
0.0011936585315613136
0.0011936576218101497
0.0011936567455880637
0.00119365589807763
0.0011936550674653838
0.0011936542645024871
0.0011936534922309375
0.001193652746126346
0.0011936520132733862
0.0011936513092086513
0.0011936506296188168
0.0011936499621656306
0.0011936493084728304
```

```
0.0011936486893746484
0.0011936480830592303
0.0011936474987434065
0.0011936469266048307
0.0011936463755846533
0.001193645834613386
0.0011936453258156567
0.0011936448160405056
0.0011936443272281317
0.0011936438558486173
0.001193643396032939
0.0011936429529034128
0.0011936425274189815
0.001193642101867987
0.0011936417026320561
0.0011936413107999668
0.0011936409276083121
0.0011936405590963028
0.0011936401987934905
0.0011936398537931833
0.0011936395140860912
0.0011936391902910063
0.0011936388726451349
0.0011936385610983452
0.0011936382634134624
0.0011936379791694202
0.0011936377003744856
0.001193637426972531
0.0011936371651871513
0.0011936369106272688
0.0011936366589421908
0.0011936364203095785
0.001193636192272569
0.0011936359628716728
0.0011936357447808303
0.0011936355366574238
0.001193635326142728
0.0011936351263073736
0.0011936349368021671
0.001193634750174238
0.001193634568214856
0.001193634389966832
0.0011936342195492564
0.0011936340534869786
0.001193633895748109
0.001193633742070261
0.0011936335907353087
0.0011936334425519074
0.00119363330209281
0.0011936331653479746
0.0011936330307045547
0.001193632898162515
0.001193632773538682
0.001193632655840162
0.0011936325364140866
0.0011936324237174134
0.0011936323086596157
0.0011936322048499953
0.0011936320993227783
0.0011936319993315501
```

```
0.001193631901533322
0.0011936318058937058
0.001193631711743295
0.0011936316239937617
0.0011936315375779042
0.0011936314531070368
0.0011936313746481798
0.001193631293321405
0.0011936312172571122
0.00119363114293677
0.0011936310703336906
0.0011936310026517888
0.001193630932803405
0.0011936308671937927
0.0011936308036691959
0.001193630744689185
0.0011936306830658606
0.0011936306263910083
0.001193630567671026
0.0011936305127456543
```

In [25]:
```
y_prediction,_=model03(xgrid',parameters03, layer_states03)


fig = Figure()
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"x", ylabel = L"f(x)")
lines!(ax,xgrid[:],y_prediction[:];color=:blue,label="prediction",linewid
scatter!(ax,xgrid[:],data03[:];color=:black,label="data")
fig
```



In [26]:
```
f03_loss(parameters03)
```

```
0.0011936305127456543
```

Now we will use the optimisation routines in Julia to train the Neural Network

In [27]:
```
callback = function (p, l)
    if length(loss_history) % 100 == 0
        println("Iteration: $(p.iter), Loss: $l")
    end
    push!(loss_history, l)
    return false
end

adtype = Optimization.AutoZygote()
optf = Optimization.OptimizationFunction(loss_fn03, adtype)
optprob = Optimization.OptimizationProblem(optf, ComponentArray(parameter

# Initialize a vector to store loss values
loss_history = Float64[]
neural_network = Optimization.solve(optprob, OptimizationOptimisers.Adam(

parameters03=neural_network.u
```

```
Iteration: 1, Loss: 0.0011936305127456543
Iteration: 101, Loss: 0.0011936305071361944
Iteration: 201, Loss: 0.001193628776664015
Iteration: 301, Loss: 0.0011936287766511797
Iteration: 401, Loss: 0.0011936287766759505
Iteration: 501, Loss: 0.001193628776669221
Iteration: 601, Loss: 0.0011936287766354029
Iteration: 701, Loss: 0.001193628776691832
Iteration: 801, Loss: 0.0011936287768013175
Iteration: 901, Loss: 0.0011936287767116392
Iteration: 1000, Loss: 0.0011936287766241454
ComponentVector{Float32}(layer_1 = (weight = Float32[0.6321452;;], bias =
Float32[6.088589f-17]), layer_2 = (weight = Float32[2.157455;;], bias = Fl
oat32[6.600071f-17]))
```

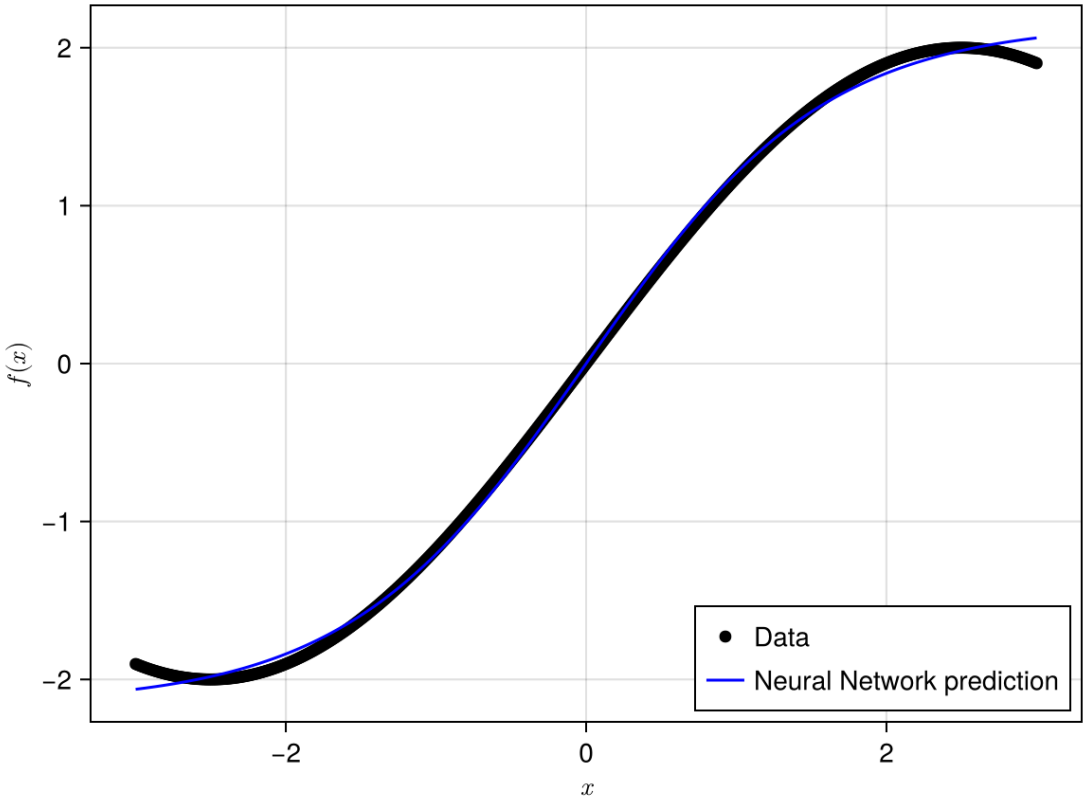
Calculate the loss for the current set of parameters

In [28]:
```
loss_fn03(parameters03, layer_states03)
```

```
0.0011936287766241454
```

Plot to see if the prediction from the new model matches the data

In [29]:
```
y_prediction,_=model03(reshape(xgrid,(1,N_SAMPLES)),parameters03, layer_s

fig = Figure()
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"x", ylabel = L"f(x)")
scatter!(ax,xgrid,data03;color=:black,label="Data")
lines!(ax,xgrid,y_prediction[:];color=:blue,label="Neural Network predict
axislegend(ax,  position = :rb,orientation = :vertical)
fig
```



Class Demo 02: Construct a single input single output Neural Network using Lux in Julia to fit the data "Data05.csv"
